

AI Security 2025–2026: What's Ready to Build but Not Yet Built

TL;DR

- The single biggest concentration of "research exists, mature product doesn't" opportunity is in **agentic / MCP security and indirect prompt-injection containment**: cheap, deterministic, model-agnostic techniques (PromptArmor LLM-as-detector, DefensiveTokens, Meta's Agents Rule of Two, lethal-trifecta taint-tracking, MCP runtime monitoring) are published and partly open-sourced, but no widely distributed developer-grade product yet wraps them cleanly.
- The architecturally strongest defenses (Google DeepMind's CaMeL, the Dual-LLM pattern, capability/taint enforcement) remain **theoretical with no production implementation as of mid-2026** — a genuinely open lane for a small team to ship the first real harness/library.
- On the "AI applied to security" side, autonomous pentest/vuln-discovery (XBOW, Big Sleep,

CodeMender) is being captured by well-funded incumbents, but **narrow, cheap wrappers** — MCP security scanners, SOC alert-triage agents, CVE-reproduction, malware-triage copilots — are still wide open for lightweight productization.

Key Findings

Security OF AI (defending models/apps). Prompt injection (direct and indirect) remains the #1 risk in the OWASP Top 10 for LLM Applications (2025) and is the root of the new OWASP Top 10 for Agentic Applications (released December 9, 2025). The field has converged on a hard truth: probabilistic detection alone fails. The OpenAI/Anthropic/Google DeepMind study "The Attacker Moves Second" (arXiv:2510.09023, Oct 2025) bypassed 12 recent defenses with attack success rate above 90% for most — prompting-based defenses hit 95–99% ASR, training-based defenses 96–100%, and human red-teaming achieved 100%. The momentum is therefore toward cheap architectural/deterministic containment. Several of these are easy to implement and not yet productized.

MCP security is the hottest, least-mature surface.

Between January and April 2026, researchers disclosed 40+ CVEs against MCP implementations; an OX Security advisory described a "by design" RCE

class in Anthropic's official SDK across Python/TypeScript/Java/Rust, with surveys showing a majority of implementations vulnerable to path traversal and injection. Open-source scanners exist (Invariant Labs' MCP-Scan, acquired by Snyk; Cisco's mcp-scanner; eSentire's MCP-Scanner) but coverage is partial and the runtime-monitoring layer is acknowledged as the gap.

AI applied to cybersecurity. Autonomous offensive AI crossed real thresholds: XBOW reached #1 on HackerOne's US leaderboard (by reputation gain, Apr–Jun 2025), submitting nearly 1,060 vulnerabilities (54 critical, 242 high, 524 medium, 65 low) and raising \$75M led by Altimeter (later \$237M total). Google's Big Sleep found real-world 0-days (e.g., CVE-2025-6965 in SQLite, found before in-the-wild exploitation); DeepMind's CodeMender autonomously patches code — per its October 2025 blog (Raluca Ada Popa & Four Flynn), "we have already upstreamed 72 security fixes to open source projects, including some as large as 4.5 million lines of code." These are capital-intensive incumbents — but the surrounding tooling (triage, reproduction, scanning) is cheap to build.

Details

Category 1 — Indirect Prompt Injection Defenses (productizable now)

PromptArmor (Shi et al., arXiv:2507.15219, July 2025). Uses an off-the-shelf LLM (GPT-4o/4.1/o4-mini) as a guardrail that detects and removes injected prompts before they reach the agent. Reported result: "using GPT-4o, GPT-4.1, or o4-mini, PromptArmor achieves both a false positive rate and a false negative rate below 1% on the AgentDojo benchmark... after removing injected prompts with PromptArmor, the attack success rate drops to below 1%." (Below 5% FPR/FNR on Open Prompt Injection and TensorTrust.)

- *Why cheap:* It's just a well-crafted system prompt around an existing LLM API — no training, no infra.
 - *Productized version:* A drop-in "injection firewall" API/SDK or MCP-proxy middleware; a one-line wrapper for LangChain/LlamaIndex tool outputs.
 - *State of implementations:* No confirmed official code repo tied to the paper (note: a separate "prompt-armor" GitHub project and a "PromptArmor" research group are unrelated). Commercial guardrails (Lakera Guard, Prompt Security, LLM Guard, Rebuff) do related detection but not this specific detect-and-remove-on-tool-output pattern as a clean developer primitive.
- Open opportunity.**

DefensiveTokens (Chen, Carlini, Sitawarin, Wagner et al., arXiv:2507.07974; ACM AISEC '25). Adds just 5

new special tokens whose embeddings are optimized for security; no model weights changed; appended only when security matters, so zero utility loss otherwise. Reported: in the largest test (>31K samples), DefensiveTokens mitigate manually-designed prompt injections to 0.24% ASR (averaged across four models, comparable to training-time defenses at 0.20–0.51% and far below test-time alternatives at >11%); for optimization-based injection, average ASR drops from 95.2% to 48.8%; on an agentic tool-calling benchmark, ASR is reduced 5x. Official open-source code at github.com/SizheChen/DefensiveToken.

- *Why cheap*: Soft-prompt tuning; trivially cheap to run; queries with/without tokens batch together with no infrastructure changes.
- *Productized version*: A hosted "defensive token pack" for open-weight models (Llama, Qwen, Mistral); a fine-tune-free hardening service for self-hosters.
- *State*: Research artifact only; needs provider to release tokens. Open for self-hosted-LLM shops.

Meta's "Agents Rule of Two" (October 31, 2025).

Deterministic design rule: until prompt injection is reliably solved, an agent "must satisfy no more than two" of three properties within a session – (A) process untrustworthy inputs, (B) access sensitive

systems/private data, (C) change state or communicate externally. If all three are required without a fresh session, the agent "should not be permitted to operate autonomously and at a minimum requires supervision — via human-in-the-loop approval or another reliable means of validation." Official source: ai.meta.com/blog/practical-ai-agent-security/. (Note: Meta edited its original diagram after pushback, replacing "safe" with "lower risk" for the two-way overlaps; it reduces severity, not risk entirely.)

- *Why cheap*: It's a policy/architecture pattern enforceable with metadata + a gating layer — no ML.
- *Productized version*: A "trifecta linter"/policy engine for agent configs and MCP tool manifests that tags each tool reads_private_data / sees_untrusted / can_exfiltrate and blocks all-three execution paths (Simon Willison and Oso both proposed exactly this taint-tracking enforcement). [Substack](#) [Oso](#) Builds directly on Willison's June 16, 2025 "lethal trifecta" framing (private data + untrusted content + external communication).
- *State*: Framework only, no enforcement product. Meta references its own open-source LlamaFirewall, Prompt Guard, Llama Guard, and Code Shield, but the Rule of Two itself is conceptual. **Very open, very cheap.**

IntentGuard (arXiv:2512.00966), InstructDetector (arXiv:2505.06311), "Attention is All You Need to Defend..." (arXiv:2512.08417), CachePrune (2504.21228), IPIGuard (2508.15310). A wave of late-2025 training-free or lightweight detection methods using instruction-following intent analysis, hidden-state/attention signals, and tool-dependency graphs. Most are cheap detectors that could become libraries; few have polished products.

CaMeL (Google DeepMind, April 2025). Capability-based architectural defense: a privileged LLM plans, a quarantined LLM handles untrusted data, a custom interpreter enforces data/control-flow policies. [SSOJet](#) Solves ~67–100% of AgentDojo attacks depending on config, at ~2.7–2.8x token cost. Crucially: **no production-grade CaMeL implementation exists as of mid-2026** [SOPHOS](#) (confirmed by Sophos and others). This is the strongest architectural pattern with zero shipping product — a flagship opportunity, though non-trivial to build (it requires a custom Python interpreter and capability-management components).

Category 2 — Jailbreak Detection & Guardrails (cheap classifiers)

"LLM Jailbreak Detection for (Almost) Free!" (Chen et al., EMNLP 2025 Findings; arXiv:2509.14558). Prepends an affirmative instruction and inspects first-

token logit confidence — near-zero added compute, no extra model. Productizable as a tiny middleware/decorator for any self-hosted model. No standalone product.

Internal-representations detectors

(arXiv:2602.11495; LVLM variant 2512.12069).

Lightweight classifiers on hidden states separate jailbreak from benign prompts cheaply. Good library/SDK candidates. (Caveat: arXiv:2504.11168 shows character-injection attacks can evade guardrail classifiers the underlying LLM still understands — detectors need ongoing hardening.)

Existing products to triangulate against: Lakera (Guard + Red; ~70 staff, Zurich/SF) — Check Point agreed to acquire for an estimated \$300M, announced September 16, 2025, expected to close Q4 2025. Prompt Security (SentinelOne, May 2025), [AppSec Santa](#) Protect AI (Palo Alto, July 2025; built on open-source ModelScan), Robust Intelligence (Cisco, 2024), HiddenLayer, plus open-source LLM Guard, NeMo Guardrails, Rebuff. The consolidation wave means **standalone, developer-first, agent/MCP-native guardrails are under-served** — incumbents are being absorbed into enterprise suites, and several have limited-to-no agentic/MCP coverage.

Category 3 — MCP & Agent Supply-Chain Security

MCP-Scan (Invariant Labs / Snyk), Cisco mcp-scanner, eSentire MCP-Scanner, MCPGuard (arXiv:2510.23673), MCP-Guard (2026). Detect tool poisoning, rug pulls, cross-origin/tool-shadowing, prompt injection in tool descriptions. [AppSec Santa](#) Hasan et al. ("MCP at First Glance") scanned 1,899 MCP servers and found 5.5% with tool poisoning; AgentSeal scanned 1,808 servers and found 66% had security findings (~33% allowing unrestricted network access). Static scanning is solved-ish; **runtime behavioral monitoring** (sequence-level attacks, post-authorization activity) is the acknowledged infrastructure gap.

- *Productized version:* An MCP runtime firewall/proxy that logs every tool call, enforces taint/trifecta policy, detects rug-pull mutations via tool-hash pinning, and does behavioral anomaly detection. A "5 runtime signals" approach (full execution traces, tool-call anomaly detection, credential-access monitoring, memory-write auditing, input screening at ingest) packaged as a lightweight sidecar.
- *State:* Pieces exist open-source; an integrated, easy runtime product is open.

Agent memory poisoning — A-MemGuard (arXiv:2510.02373). First defense for LLM-agent memory: consensus-based validation (parallel reasoning paths) + a dual-memory "lessons" structure,

cutting attack success rates by over 95% with minimal utility cost. Memory poisoning (MINJA — >95% injection success, ~70% attack success; MemoryGraft; Palo Alto Unit 42 PoC) persists across sessions and evades single-detector audits (which miss ~66% of poisoned entries).

- *Productized version*: A "memory firewall" wrapper for mem0/LangGraph/vector stores with provenance tagging, trust scoring, temporal decay. No mature product. **Open**.

RAG poisoning defenses — RAGForensics (2504.21668, code available), TrustRAG, RobustRAG, ReliabilityRAG (2509.23519), RAGShield (2604.00387). Traceback/attribution of poisoned documents, provenance verification, knowledge-expansion. (Underlying attacks PoisonedRAG and CorruptRAG show simple paraphrasing/instructional defenses are insufficient.) Cheap to wrap around existing RAG pipelines as a scanning/attribution microservice. Largely unproductized.

Category 4 — Watermarking & Provenance

SynthID-Text (Google DeepMind, Nature 2024; open-sourced). Production-ready logits-processor watermarking with efficient detection (without using the underlying LLM), integrated in Hugging Face Transformers with a Bayesian detector. Cheap to

adopt for any self-hosted model; modifies only sampling, not training.

- *Productized version:* A provenance/watermark-as-a-service for platforms generating AI text (helps with EU AI Act labeling); a detection API. Mostly a Google-controlled ecosystem, but integration/compliance tooling for smaller providers is open. (Caveat: text watermarks are fragile to paraphrasing/back-translation.)

Category 5 – AI Applied to Cybersecurity

Autonomous SOC / alert triage. CORTEX

(arXiv:2510.00311) and commercial entrants (Conifers CognitiveSOC, Dropzone, Torq Socrates, CrowdStrike Charlotte Agentic SOAR, Splunk ES) show multi-agent triage works. Incumbents are crowding the high end, but a cheap, narrowly scoped Tier-1 triage agent (SIEM/EDR-integrated, LLM + deterministic logic) is buildable by a small team; the moat is integrations, not the AI. Note Gartner's December 2024 prediction "There Will Never Be an Autonomous SOC" — position as augmentation.

LLM vulnerability discovery & reproduction. CVE-

Genie (arXiv:2509.01835) "successfully reproduces approximately 51% (428 of 841) CVEs published in 2024-2025, complete with their verifiable exploits, at an average cost of \$2.77 per CVE" — across 267

projects, 141 CWEs, 22 languages. VulnInstruct found a previously unknown high-severity bug (CVE-2025-56538); CVE-Bench benchmarks repair (best SWE-agent repaired only 21%). These are cheap, API-driven pipelines — a "CVE-to-PoC" or "patch-suggestion" developer tool is feasible without frontier-lab resources.

Malware analysis / reverse engineering copilots.

AgentRE-Bench (zero Python deps, bring-your-own-API-key), TraceRAG (Android malware explanation), LLM-for-software-security surveys (arXiv:2504.07137). A binary/script triage copilot that auto-generates YARA rules and explains decompiled code is a cheap, concrete product — current state of the art is "co-pilot," not autonomy.

AI pentest wrappers. XBOW (\$237M raised) [AppSec Santa](#) and Anthropic's internal tooling dominate full autonomy. PentestGPT (MIT license, ~86.5% on XBOW's validation benchmark at ~\$1.11/solved benchmark) shows cheap open baselines exist. A vertical, scoped scanner (e.g., MCP-server pentest-as-a-service, or LLM-app DAST) is a realistic wedge.

Recommendations

Tier 1 — Build now (lowest effort, most open):

1. MCP runtime security proxy / "trifecta linter."

Combine Meta's Rule of Two + Willison/Oso taint-tracking + tool-hash pinning (rug-pull detection) + full tool-call tracing. Ship as an open-core MCP proxy + SaaS dashboard. Static scanning is commoditized; runtime is the gap. Effort: low-medium. Opportunity: large and timely (40+ MCP CVEs in early 2026).

2. Indirect-injection "firewall" SDK implementing

PromptArmor's detect-and-remove on tool/RAG outputs, plus the cheap jailbreak first-token detector. One-line wrappers for LangChain/LlamaIndex/MCP. Effort: low. Opportunity: high; no clean developer primitive exists.

3. Agent memory firewall (A-MemGuard-style consensus validation + provenance tagging) for mem0/LangGraph. Effort: low-medium. Opportunity: high; no product.

Tier 2 — Build with moderate effort:

4. RAG poisoning scanner/attribution microservice (RAGForensics + perplexity/provenance checks).

5. CVE-to-PoC / reproduction pipeline (CVE-Genie-style) as a dev/security tool at ~single-dollar cost per CVE.

6. DefensiveToken hardening service for open-weight self-hosters.

Tier 3 — Ambitious / higher moat:

7. **First production CaMeL/Dual-LLM harness** — the strongest architectural defense with no shipping implementation. Hard but flagship.

Benchmarks/thresholds that change the plan: If a major agent framework (LangChain, OpenAI Agents SDK, Anthropic) ships native Rule-of-Two/taint enforcement or a CaMeL-style harness, Tier-1 #1 and Tier-3 #7 shrink fast — move first. If Snyk/Cisco extend MCP-Scan into full runtime monitoring, the MCP-proxy window narrows; differentiate on developer experience and open-core. Watch OWASP Agentic Top 10 adoption and the NIST/CAISI agent-eval standards as demand signals; a first widely-publicized "lethal trifecta" enterprise breach (not yet occurred as of mid-2026) would massively accelerate buyer urgency.

Caveats

- Several "results" are from arXiv preprints not yet peer-reviewed; numbers (e.g., ASR reductions) are self-reported on specific benchmarks (often AgentDojo) and may not generalize. "The Attacker Moves Second" (arXiv:2510.09023) shows detector-based defenses are routinely bypassed by adaptive attacks at >90% ASR — favor

architectural/deterministic approaches over pure classifiers.

- The vendor-acquisition wave (Lakera → Check Point ~\$300M Sept 2025; Prompt Security → SentinelOne; Protect AI → Palo Alto; Robust Intelligence → Cisco; Invariant Labs → Snyk, all 2024–2025) means today's "open" gaps can close quickly; speed matters.
- Some forward-looking claims in sources use speculative language ("could," "future") — autonomous SOC and full-autonomy pentest remain partly aspirational; Gartner explicitly predicted there will never be a fully autonomous SOC.
- Dates/CVE counts for 2026 come from vendor blogs and aggregators; treat specific 2026 CVE tallies as indicative, not audited.
- "Cheap to build" refers to engineering effort and inference cost, not go-to-market: in AI security the durable moat is integrations, distribution, and trust, not the core algorithm — most of the techniques above are individually reproducible in days to weeks.